

category: dotnet-general

tags: [cross-platform, linux, projectreference, hintpath, system.drawing, system.data.oledb, compile-remove]

severity: high

created: 2026-07-02

updated: 2026-07-02

project-origin: iPGSv4 (CCU — branch ccu-avalonia)

BaseLIB ProjectReference bị broken: switch sang pre-built DLL HintPath

Tình huống gặp phải

Worktree Avalonia build (ccu-avalonia) có <ProjectReference> trỏ vào 0.BaseLIB\Kztek.Tool\... — BaseLIB source bị broken vì Kztek.Object.Entity (dependency của Kztek.Tool) đã xóa các enum mà Kztek.Tool dùng

(EmSystemAction, EmSystemActionType, ...). Kết quả: mọi project reference đến Kztek.Tool đều fail với 62+ errors,

kể cả IPGS.Object, IPGSv4, IPGS.Cash.

Triệu chứng / Lỗi

```
error CS0234: The type or namespace name 'Object' does not exist in the namespace 'Kztek'
error CS0246: The type or namespace name 'EmSystemAction' could not be found
62 Error(s)
```

Lỗi xuất hiện trong Kztek.Tool.csproj (BaseLIB), không phải trong project đang build.

Nguyên nhân gốc rễ

Kztek.Tool (BaseLIB) → ProjectReference → Kztek.Object.Entity (BaseLIB) → missing enum types.

BaseLIB repo đang ở branch khác / commit mà Kztek.Object.Entity bị break.

Không có quyền fix BaseLIB từ worktree project.

Giải pháp

Bước 1 — Xác nhận pre-built DLL tồn tại

```
find "E:\KZTEK\Code_Git\1.Window\0.BaseLIB\Kztek.Tool" -name "Kztek.Tool.dll"
2>/dev/null
```

Kết quả: DLL pre-built tồn tại ở bin\x64\Release\netstandard2.1\Kztek.Tool.dll (build từ lúc BaseLIB còn tốt).

Bước 2 — Switch từ ProjectReference sang Reference + HintPath

```
<!-- Trước (broken): -->
<ProjectReference
Include="..\..\..\..\..\0.BaseLIB\Kztek.Tool\...\Kztek.Tool.csproj" />

<!-- Sau (dùng pre-built DLL): -->
<Reference Include="Kztek.Tool">

<HintPath>..\..\..\..\..\0.BaseLIB\Kztek.Tool\Kztek.Tool\1.Source\Kztek.Tool\bin\x64
\Release\netstandard2.1\Kztek.Tool.dll</HintPath>
</Reference>
```

Lưu ý quan trọng: Dùng `bin\x64\Release` (không phải `Debug`) — stable hơn và thường là bản có sẵn.

Bước 3 — Với project reference không dùng trong compiled code: xóa hẳn

Ví dụ: Kztek.Api chỉ được `using Kztek.Api;` trong `Form1.cs` đã excluded → xóa ProjectReference khỏi csproj hoàn toàn.

```
<!-- Xóa: -->
<ProjectReference Include="..\Kztek.Api.csproj" />
```

Khi nào dùng pre-built DLL Reference vs ProjectReference

Tình huống	Chọn
---	---
Source builds clean, không bị broken	ProjectReference (rebuild tự động khi code đổi)
Source broken do dependency bên ngoài scope sửa	<Reference> + HintPath đến pre-built DLL
Loại bỏ hoàn toàn dependency	Xóa reference nếu type không dùng trong compiled code

System.Drawing trên cross-platform .NET 8

Hay nhầm lẫn — hai nhóm type khác nhau:

Type	Từ đâu	Cross-platform?
---	---	---
Point, Size, Rectangle, Color, PointF	System.Drawing.Primitives	Có — không cần package
Image, Bitmap, Graphics, Pen, Brush, Drawing2D.*	System.Drawing.Common	Không — Windows-only từ .NET 7+

Quy tắc: `using System.Drawing;` OK nếu chỉ dùng Point/Size/Rectangle. Fail trên linux nếu dùng Image/Bitmap/Graphics.

System.Data.OleDb

`System.Data.OleDb` là Windows-only (Microsoft Jet OLEDB Provider chỉ có trên Windows).

- Nếu project target `net8.0` (không `-windows`) và cần build trên linux-x64 → PHẢI loại.
- Fix: <Compile Remove> file dùng OleDb + xóa <PackageReference Include="System.Data.OleDb">.

- Nếu class còn được type-reference từ file khác (ví dụ: `StaticPool.Mdb`) → rewrite class để không dùng OleDb.

Khi `<Compile Remove>` một class: cascade check bắt buộc

Sau khi thêm `<Compile Remove="Foo.cs" />`, PHẢI grep tìm tất cả `new Foo(, Foo , : Foo` trong các file CÒN LẠI trong build:

```
grep -rn "HikCameraControllerv2" --include="*.cs" IPGS.Object/ 2>/dev/null
# → tìm thấy CameraObject.cs dùng new HikCameraControllerv2(this) → phải fix
CameraObject.cs
```

Không làm bước này → build fail với CS0246 ("type or namespace name could not be found").

Lỗi tiềm ẩn bị che khuất bởi dependency failure

Khi dependency fail với 62 errors, compiler dừng sớm → không phát hiện lỗi ở layer cao hơn.

Sau khi fix dependency, các lỗi mới lộ ra, ví dụ:

- Stray identifier `ucbaseLaneOut` trong `Program.cs` → CS1002 syntax error (chỉ lộ sau khi Kztek.Tool được fix).

Luôn kiểm tra lại từng project sau khi fix dependency.

Áp dụng lại

- Khi build fail với errors từ BaseLIB ProjectReference → check nếu pre-built DLL tồn tại trong `bin/` → switch sang HintPath
- Trước khi `<Compile Remove>` bất kỳ class nào → grep `new ClassName\|ClassName` trong toàn bộ project còn compile
- Khi fix dependency lớn → rebuild từng project riêng lẻ để phát hiện lỗi cascade từng bước

Chú ý / Cạm bẫy

- ⚠ DLL `netstandard2.1` dùng được cho cả win-x64 lẫn linux-x64 compile (khác với runtime)
- ⚠ Khi dùng pre-built DLL Reference: compiler chỉ cần DLL metadata — không cần transitive deps của DLL đó ở compile time
- ⚠ `System.Data.OleDb` phải loại cả package lẫn code; chỉ loại code mà giữ package = build warning, không fail build nhưng vẫn Windows-only
- ⚠ `bin\x64\Release` path là tương đối — portable nếu repo được checkout ở cùng cấu trúc thư mục

Tham chiếu

- Project: `iPGSv4 / IPGS.Object, IPGSv4, IPGS.Cash` (branch `ccu-avalonia`)
- Commit: `922cc85` — "fix(cross-platform): eliminate Windows-only deps from IPGS.Object, fix hardcoded paths, delete orphan views"
- Files đã sửa: `IPGS.Object.csproj, MDB.cs, Edge.cs, ICameraController.cs, StaticPool.cs, CameraObject.cs, IPGSv4.csproj, IPGS.Cash.csproj, Program.cs`